

Formularea problemei

1. INTRODUCERE

Prezentul document urmărește definirea publicului-țintă și a actorilor unei aplicații software. Aplicația analizată este un joc video 2D realizat în Godot. Acesta este multiplayer și colaborativ, bazat pe un model economic dinamic. Jucătorii controlează economii locale ce alcătuiesc un ecosistem economic global al cărui echilibru este influențat de decizii individuale. Scopul jocului este menținerea echilibrului prin lucru în echipă.

Documentul își propune să ofere o imagine de ansamblu asupra modului în care diferitele tipuri de utilizatori interacționează cu platforma, precum și asupra responsabilităților, limitărilor și permisiunilor atribuite fiecăruia. Această definire reprezintă o componentă centrală în proiectarea sistemului multi-utilizator, asigurând compatibilitatea dintre cerințe, arhitectura tehnică și experiența utilizatorilor.

2. PUBLICUL-ȚINTĂ

Aplicația urmărește o audiență diversă, atât din sfera de gaming, cât și din mediul academic. Principalele categorii de utilizatori sunt:

- Jucători interesați de jocuri de strategie. Aceștia reprezintă segmentul principal de audiență. Sunt utilizatori care apreciază jocuri de strategie economică, management de resurse și simulări în timp real. Aceștia caută o experiență interactivă, accesibilă și satisfăcătoare, fără a necesita o curbă de învățare foarte abruptă.
- Studenți și profesori din domeniul economic. Aplicația urmărește simularea unor concepte economice elementare, precum cererea, oferta și inflația, într-un mod vizual și interactiv. Aceasta are, astfel, un potențial ridicat pentru a fi o unealtă educativă.
- Jucători interesați de jocuri colaborative. Componenta multiplayer în timp real și prezența unui obiectiv comun pentru jucători face aplicația atractivă pentru comunitățile pasionate de jocuri ce implică cooperarea și lucrul în echipă. Prin acest aspect, aplicația prezintă potențial de interes inclusiv pentru amatorii de jocuri de masă.

3. ACTORII SISTEMULUI

Sistemul urmărește trei actori principali, fiecare având roluri și permisiuni proprii.

3.1. Jucător

Acesta este actorul principal al sistemului. Controlează o economie individuală într-o sesiune multiplayer și poate întreprinde următoarele acțiuni:

- Autentificare / login
- Intrarea într-o sesiune de joc, prin matchmaking sau prin crearea propriului lobby

- Gestionarea economiei proprii, prin construirea / îmbunătățirea facilităților de producție și prin optimizarea producției și a vânzării
- Comerț, prin crearea de oferte și acceptarea / refuzul ofertelor altor jucătorilor
- Monitorizarea indicatorilor economici: inflație, cerere și ofertă
- Comunicarea cu ceilalți jucători prin chat
- Părăsirea sesiunii de joc

3.2. Jucător bot

Actor software, controlat de server. Acesta menține piața dinamică prin comerț cu alți boți, oferă jucătorului posibilitatea de a juca o sesiune „solo” și are implementate mecanisme care să prevină exploatarea sa. Activitățile pe care le poate întreprinde sunt:

- Producție limitată, artificială de resurse
- Comerț cu ceilalți jucători în mod controlat
- Evaluează gradul de încredere al celorlalți jucători pentru a nu fi exploatat de cei ce urmăresc destabilizarea economiei

3.3 Sistemul (Serverul)

Actor tehnic ce cu următoarele atribuții:

- Gestionarea logicii de joc, prin procesarea tick-urilor economice și prin calcularea și recalcularea indicatorilor economici
- Administrarea aspectelor multiplayer: conectarea și deconectarea jucătorilor, sincronizarea stărilor, gestionarea situațiilor de conexiune slabă
- Asigurarea persistenței datelor
- Gestionarea boților

3.4 Matricea permisiunilor

Activitate / Actor	Jucător	Bot	Server
Autentificare	Da	-	-
Creare sesiune	Da	Nu	Da
Intrare în sesiune	Da	Da	Da
Producție resurse	Da	Da (limitat)	Nu
Comerț P2P	Da	Da (limitat)	Nu
Acces piață globală	Da	Da (limitat)	Administare
Activarea finalului „egoist”	Da	Nu	Nu
Vizualizare indicatori	Da	Da	Da

economici			
Persistență a datelor	Nu	Nu	Da
Gestionare boți	Nu	-	Da

4. CERINȚE DE PERFORMANȚĂ ȘI PARALELISM

4.1. Reacție în timp real

Având în vedere că jocul funcționează cu tick-uri rapide și tranzacții economice multiple, aplicația trebuie să asigure:

- latență medie sub 150 ms între client și server
- rate stabile de actualizare a stării (broadcast WebSocket) la 10–20 actualizări pe secundă (în funcție de încărcare)
- procesare server-side în maxim 50 ms per eveniment

Frontendul Godot va procesa evenimentele în cadrul unui loop, iar backend-ul trebuie să țină pasul prin structurarea logicii într-un mod event-driven.

4.2. Paralelism

Backend-ul implementat în Node.js trebuie să:

- gestioneze simultan multiple conexiuni WebSocket active în cadrul unei sesiuni
- proceseze în paralel tranzacțiile economice prin intermediul mecanismelor asincrone (Promises, event loop)
- evite blocările (non-blocking I/O)

4.3. Consistență în timp real

Server-ul este autoritativ — deciziile economice se iau exclusiv la nivelul backend-ului. Cerința de consistență este de tip „consistență eventuală cu constrângeri”:

- toate operațiunile critice (comerț, procesare resurse) trebuie să fie confirmate prin server
- stările afișate trebuie să reflecte la maximum 100–200 ms de la realitatea serverului

5. CERINȚE DE SECURITATE

5.1. Autentificare

- autentificare prin JWT (JSON Web Token),
- expirare token,
- reîmprospătare token automat (refresh flow),
- validare strictă a identității jucătorilor.

5.2. Autorizare

- acces diferențiat în funcție de rol: Player, Bot, Server,
- verificare server-side a fiecărei acțiuni ce poate modifica state-ul economiei,
- prevenirea tranzacțiilor invalide sau a manipulării pieței prin mesaje falsificate.

5.3. Criptarea comunicării

- toate comunicările sunt transmise exclusiv prin HTTPS
- datele sensibile nu circulă niciodată în clar.

5.4. Protecții suplimentare

- rate limiting pe acțiuni economice (prevenire spam trade),
- validarea fiecărei acțiuni pe server (sanitizare input),
- protecție împotriva sincronizării neautorizate (ex: trimiterea de state manual modificate în Godot).

5.5. Integritate

Toate tranzacțiile sunt logate în baza de date, cu timestamp-uri exacte. Acest lucru permite auditarea completă a jocului și prevenirea fraudelor intenționate.

6. CERINȚE DE SCALABILITATE

6.1. Scalabilitate verticală

Sistemul trebuie să poată rula în condiții optime pe o singură instanță de server:

- CPU multicore,
- memorie minimă 4–8 GB.

6.2. Scalabilitate orizontală (potențială)

Arhitectura Node.js permite, dacă devine necesar:

- clustering (Node.js Cluster Module),
- load balancing,
- separarea serviciilor (economy engine, trade service, matchmaking).

6.3. Scalabilitatea bazei de date

PostgreSQL oferă:

- indexing,
- query optimizat,
- replicare la nivel citire/scriere

- tranzacții ACID pentru consistență economică.

6.4. Scalabilitatea numărului de jucători

Sistemul trebuie să suporte:

- minim 20–30 de jucători activi per sesiune,
- conectări simultane la nivel global fără degradarea severă a performanțelor.

7. CERINȚE DE MENTENABILITATE

7.1. Cod modular

Arhitectura trebuie să fie structurată pe module distincte:

- Authentication Module,
- WebSocket Hub,
- Trade Engine,
- Economy Engine,
- Bot Manager,
- Persistence Layer.

Această structură facilitează izolarea bug-urilor și extinderea funcționalităților.

7.2. Respectarea principiilor de proiectare

- SOLID (în special responsabilitatea unică și separarea interfețelor),
- DI (Dependency Injection în backend),
- DRY (evitarea duplicării logicii economice).

7.3. Testare

- unit testing pentru componente critice (trade, inflație),
- testare de integrare între client și server,
- testare de performanță pentru WebSockets.

7.4. Documentație tehnică

Codul sursă trebuie însoțit de:

- comentarii,
- README-uri per modul,
- diagrame arhitecturale (C4),
- ghid de instalare și configurare.

7.5. Ușurința actualizărilor

Serverul trebuie proiectat astfel încât:

- actualizările să poată fi aplicate fără întreruperea sesiunilor active (rolling updates),
- schimbările în schema DB să fie gestionate prin migrații.

8. PLATFORME ȘI BROWSERE SUPORTATE

8.1. Platforme client

Godot permite exportul pe:

- Windows,
- Linux,
- macOS.

Cu efort redus, se pot suporta:

- Android,
- iOS.

Pentru proiectul de licență platforma principală va fi Windows.

8.2. Platforme server

Backend-ul Node.js este compatibil cu:

- Linux,
- Windows Server,
- macOS (dezvoltare, nu producție).

8.4. Compatibilitate rețea

Sistemul trebuie să funcționeze în:

- rețele LAN,
- conexiuni Wi-Fi cu latență medie,
- conexiuni mobile 4G/5G (depinde de stabilitatea WebSocket).